

Vulcan, project specification (Track 2, Private AI Agents)

Submission requirement: project specification document with scenarios, architecture, capabilities and optimization details.

1. Scenario and users

Developers working on private or regulated codebases cannot send source code to cloud LLM APIs. Vulcan is a developer-productivity agent that runs entirely on a local AMD Radeon GPU: codebase Q&A with citations, bug localization, test execution and code edits, with zero data egress.

2. Architecture

```
user — CLI (typer) — Agent (ReAct loop, JSON tool protocol)
                        |
                        |— RAG index   SQLite + cosine, embeddings (2.1)
                        |— Tools       search_code, read_file, grep,
                        |               run_cmd*, write_file
                        |— Memory      durable per-project notes
                        |— LLM client — OpenAI-compatible endpoint
                                    |
                                    |— vLLM on ROCm / Radeon GPU

* run_cmd is allowlisted; all file access is path-jailed to the indexed root.
```

2.1 Where each half actually runs

Stated precisely, because the privacy claim depends on it and it is easy to overstate.

`vllm serve Qwen/Qwen3-8B` starts with `task=generate`. The resulting server advertises `/v1/chat/completions`, `/v1/completions`, `/v1/responses`, `/v1/messages`, `/tokenize` and `/v1/models`, and **no** `/v1/embeddings`; that route returns 404. Confirmed against the live instance rather than assumed.

Serving embeddings from vLLM requires a second process started with `--task embed` against an embedding model, and Radeon Cloud permits one active instance per account. So in the configuration measured here:

stage	runs on
agent reasoning and generation	Radeon GPU, vLLM on ROCm
RAG embeddings at index time	local embedding model
retrieval, chunking, tool execution	local CPU

Both endpoints are ones the operator controls, so the no-egress property holds: no source code reaches a third party. But only generation is GPU-served in this setup, and the project does not claim otherwise.

Pointing `VULCAN_EMBED_MODEL` at an `--task embed` vLLM instance moves embeddings onto the GPU too, at the cost of a second instance.

2.2 Model choice and deployment plan

Generation model: `Qwen/Qwen3-8B`. Chosen for three reasons specific to this workload. It is small enough to serve from a single consumer Radeon at bf16 (7.67 GiB of weights, leaving room for KV cache at `--gpu-memory-utilization 0.92`), its 40960-token context comfortably holds retrieved code chunks plus a ReAct scratchpad, and it follows the strict JSON tool protocol in `vulcan/agent.py` reliably enough to drive the loop without native tool-calling support.

Qwen3 reasons by default, which is the wrong trade here and is disabled per request; see section 4.3.

Embedding model: `mxbai-embed-large`, served locally. Section 2.1 explains why it cannot share the Radeon instance.

Deploying on the Radeon (ROCm). Image `vllm-dev:rocm7.2.1-navi-ubuntu22.04-py3.10-pytorch_2.9-vllm_0.16.0`, one GPU:

```
vllm serve Qwen/Qwen3-8B --host 0.0.0.0 --port 8000 --gpu-memory-utilization 0.92
```

Cold start is roughly four minutes: weights load in about 55s, then `torch.compile` builds and caches the graphs. Subsequent starts reuse that cache and load weights in about 5s. Point the agent at it:

```
export VULCAN_BASE_URL=https://<host>/spaces/<instance-id>/8000/v1
export VULCAN_API_KEY=<per-instance key>
export VULCAN_MODEL=Qwen/Qwen3-8B
export VULCAN_ENABLE_THINKING=false
export VULCAN_EMBED_BASE_URL=http://localhost:11434/v1
```

No code change is required: `vulcan/llm.py` speaks OpenAI-compatible chat against any `base_url`. The full operational runbook, including the launch rate limits and boot failure modes actually hit during development, is in `radeon-deploy.md`.

Deploying locally (no GPU). Ollama serves both halves from one endpoint, which is the development configuration and the laptop baseline in section 4:

```
ollama pull qwen3:4b-instruct && ollama pull mxbai-embed-large
export VULCAN_BASE_URL=http://localhost:11434/v1
export VULCAN_MODEL=qwen3:4b-instruct
```

3. Capabilities

- Multi-turn codebase Q&A with file:line citations
- Autonomous tool use: semantic search, file reading, grep, test runs, edits
- Persistent memory across sessions per project
- Backend-agnostic: same agent on Ollama, llama.cpp or vLLM-ROCm

4. ROCm optimization

4.1 Measurement methodology

Every number below comes from `vulcan bench`, which streams three fixed prompts (short, medium, long) and records time-to-first-token and generation rate, taking the median of 5 repeats. The harness is the same code on both platforms, so the only variable is the backend.

Two honesty caveats that shape how the numbers should be read:

- **The Radeon runs are measured over an HTTPS proxy, not on the box.** The instance's SSH port is not reachable from the client network, so the client talks to vLLM through the Radeon Cloud spaces proxy. Median round-trip to that proxy was measured at **0.216s** immediately before the run, and that RTT sits inside every reported TTFT. GPU-side TTFT is therefore roughly `reported - 0.216`.
- **"Tokens/sec" is really content-deltas/sec.** The harness counts streamed SSE deltas, not tokenizer tokens. Deltas can coalesce in transit, so the proxied Radeon figure is a slight undercount relative to the local run.

The link to a remote GPU is not reliable, and the agent had to be hardened for it rather than assumed away. During a recorded demo run the proxy dropped mid-request with `ConnectError: [SSL: UNEXPECTED_EOF_WHILE_READING]`, killing an agent turn outright while the identical endpoint served a benchmark seconds later. `LLM.chat` now retries transport failures three times with linear backoff, and a stream is only retried while nothing has been emitted, so a half-delivered answer is returned truncated rather than silently duplicated.

Cold start matters and is reported rather than hidden: the first two repetitions of each prompt carry cache-warming cost, and the run is only stable from roughly the third repetition. Taking the median of 5 keeps a single cold outlier from dominating while still not discarding it.

4.2 Optimization axes

Single-stream tokens/sec is the number everyone reports and it is the least interesting one for an agent. A ReAct loop issues many short steps, often several at once, over a context stuffed with retrieved code. So Vulcan measures three axes, each a separate reproducible command:

axis	command	why it matters for an agent
decode	<code>vulcan bench</code>	raw generation speed, the usual headline
concurrency	<code>vulcan bench-concurrency</code>	agent fleets issue parallel steps; does the backend batch or collapse?
prefill	<code>vulcan bench-prefill</code>	RAG stuffs context, so TTFT against input length is the latency users feel

Concurrency is where the hardware argument actually lives

Measured on the laptop (Ollama, `qwen3:4b-instruct`):

concurrent requests	aggregate	per request	median TTFT
1	28.4 chunks/s	28.4	4.36s
2	20.0 chunks/s	10.0	42.21s

Adding a second concurrent request made **aggregate** throughput *fall* and pushed TTFT out by nearly 10x. Ollama serves these without continuous batching, so the second request does not share the GPU, it queues behind the first. Higher levels were not run because the curve was already inverted.

That is the honest case for the Radeon: not "the GPU is faster at one stream", but "vLLM's continuous batching keeps the curve going the right way while the laptop's inverts". Section 4.3 has the matching Radeon curve.

A note on model parity

The Radeon runs `Qwen/Qwen3-8B` and the laptop runs `qwen3:4b-instruct`. These are not the same weights, so absolute decode numbers are not a clean hardware isolation and are not presented as one. The concurrency *shape*, whether aggregate throughput rises or falls as load increases, is a property of the serving stack rather than the parameter count, and the Radeon is carrying the larger model while doing it.

An apples-to-apples run was attempted by re-serving `Qwen/Qwen3-4B-Instruct-2507` on the Radeon to match the laptop weights. It loaded (7.67 GiB) and then died twice during `torch.compile`, with the instance reaped and logs cleared, during an announced platform maintenance window. Rather than report a number that could not be reproduced, the mismatch is disclosed here.

4.3 Results

All four tables below regenerate from committed JSON with `vulcan bench-compare`.

Decode, single stream

```
vulcan bench-compare bench-results/ollama-m4air-qwen3-4b.json bench-results/radeon-vllm-qwen3-8b-nothink.json
```

case	laptop 4B ttft (s)	radeon 8B ttft (s)	laptop chunks/s	radeon chunks/s	speedup
short	0.2	0.306	32.3	24.2	0.75x
medium	0.225	0.292	20.2	24.0	1.19x
long	0.853	0.422	15.2	23.9	1.57x

The Radeon loses the short prompt and wins the long one, while carrying twice the parameters. The reason is visible in the shape: laptop throughput decays as the prompt grows (32.3 to 15.2) whereas the Radeon holds flat (24.2 to 23.9). Long-prompt TTFT halves, 0.853s to 0.422s, and that figure still has the proxy round-trip inside it.

Concurrency, the decisive axis

```
vulcan bench-compare bench-results/ollama-m4air-concurrency.json bench-results/radeon-vllm-concurrency.json
```

concurrent	laptop ttft (s)	radeon ttft (s)	laptop chunks/s	radeon chunks/s	speedup
1	4.359	1.486	28.4	23.5	0.83x
2	42.212	1.248	20.0	42.9	2.15x
4	n/a	0.553	n/a	87.0	n/a
8	n/a	0.411	n/a	179.2	n/a

Read the two curves rather than any single cell:

- **Radeon:** 23.5 to 179.2 chunks/s across an 8x load increase, a **7.63x scaling factor, 95% efficiency**. Per-request rate barely moves (23.5 to 22.4). Median TTFT *improves* 3.6x. Wall clock for the whole batch stays around 57s whether it is serving 1 request or 8.
- **Laptop:** adding a single extra request *reduces* aggregate throughput to 0.70x and pushes TTFT from 4.36s to 42.21s. Levels above 2 were not run because the curve had already inverted.

At two concurrent requests the Radeon answers in 1.2s where the laptop takes 42.2s, a **34x** difference in the latency a user actually waits. This is the real argument for the GPU, and it is invisible to any single-stream benchmark.

Prefill, the context budget

input words	radeon ttft (s)
128	0.804 (cold start)
512	0.316
2048	0.399
8192	2.091

Context is close to free up to ~2048 words, then 4x more context costs 5x the TTFT. That is a direct instruction to the RAG layer: keep retrieved context under about 2000 words and TTFT stays under half a second.

Thinking mode, a serving-config win

mode	deltas generated	wall clock	delta rate
thinking (Qwen3 default)	4058	170.9s	23.8/s
<code>enable_thinking: false</code>	1168	49.0s	24.0/s

Identical generation rate. Thinking emits ~3.5x more tokens for the same task, so per-step latency falls by the same factor. Reproducible from the CLI with `VULCAN_ENABLE_THINKING=false`.

4.4 Remaining sweeps

- ☐
Quantization sweep: fp16 vs int8/awq, quality vs speed
- ☐
vLLM tuning: gpu-memory-utilization, max-num-seqs, chunked prefill
- ☐
Embedding throughput on GPU vs CPU
- ☐
Before/after charts for the demo video

5. Demo video outline (3 to 5 min)

1. Problem: private code, no cloud (20s)
2. Index a real repo live, show GPU utilization (40s)
3. Multi-turn session: locate a bug, run tests, fix, re-run (2 min)
4. Benchmark screen: optimization journey on Radeon (1 min)
5. Architecture slide and close (30s)